

Agentic MLIR-to-QIR Translation with Verification: A Hybrid Deterministic and LLM Approach

Sharmin Afrose, Vicente Leyton-Ortega, Narasinga Rao Miniskar, Elaine Wong, Travis S. Humble

Oak Ridge National Laboratory, Oak Ridge, TN, USA

{afroses, leytonortheva, miniskarnr, wongey, humblets}@ornl.gov

Abstract—We present a hybrid system for translating MLIR quantum circuits to QIR that combines deterministic compiler techniques with large language model (LLM) agents. The system implements three translation paths: deterministic for recognized dialects (Catalyst and Quake), fully agentic for unseen dialects, and a repair path where LLM agents correct deterministic output that fails verification. A dual-backend verification pipeline independently executes source MLIR and generated QIR on separate simulators, comparing output distributions via total variation distance (TVD). A guardrail-assembler pattern recovers valid QIR from structurally incomplete LLM output, and context engineering replaces retrieval-augmented generation for smaller models. On a 39-circuit benchmark spanning three dialects (including an unseen fault-tolerant quantum computing dialect) and scaling from 5 to 100 qubits, the deterministic path achieves 98.6% gate-level correctness (71/72 circuit-dialect pairs) with TVD similarity averaging 97.9–99.3% across simulable circuits, and cross-dialect evaluation on 14 matched Catalyst–Quake pairs confirms gate-level equivalence. For the unseen FTQC dialect, the agentic path produces valid logical-level QIR on the first iteration, demonstrating the system’s extensibility to dialects beyond its deterministic parser.

Index Terms—quantum intermediate representation, MLIR, QIR, large language models, multi-agent systems, quantum circuit translation, verification

I. INTRODUCTION

Quantum computing is undergoing rapid diversification in both hardware platforms and software toolchains [1], [2]. As hardware vendors and research groups introduce competing frameworks, including IBM Qiskit [3], Google Cirq [4], Microsoft Q# [5], Xanadu PennyLane [6], and NVIDIA CUDA Quantum [7], the quantum software stack has grown increasingly fragmented. Each framework defines its own intermediate representation (IR) for expressing quantum circuits, creating a significant interoperability barrier: circuits authored in one framework cannot be readily executed on another platform’s backend without manual or semi-automated translation [8].

The Multi-Level Intermediate Representation (MLIR) framework [9], originally developed to address the “end of

Moore’s law” challenges in classical compiler infrastructure, has emerged as a unifying substrate for quantum compilation. Several quantum projects have adopted MLIR’s extensible dialect mechanism to define domain-specific representations: PennyLane Catalyst [10] introduces a quantum dialect for just-in-time compilation of hybrid quantum-classical programs, while CUDA Quantum defines the Quake dialect for GPU-accelerated quantum simulation [7]. McCaskey and Nguyen [11] first demonstrated the viability of MLIR dialects for quantum assembly languages, and recent work by Hopf et al. [12] has explored integrating multiple quantum tools within the MLIR ecosystem.

On the execution side, the Quantum Intermediate Representation (QIR) [13], built atop LLVM IR [14], has gained traction as a portable compilation target. QIR defines a standard set of quantum intrinsics (qubit allocation, gate application, measurement, and result recording) that multiple simulators and hardware backends can consume, making it a natural hub for cross-framework interoperability. However, bridging the gap between MLIR quantum dialects and QIR currently requires hand-written compiler passes for each source dialect, an approach that scales poorly as new dialects appear [8], [12].

Recent advances in large language models (LLMs) have shown strong competence in code generation [15], [16] and code translation [17] tasks. Specialized models such as Code Llama [18] and GPT-4 [19] have shown the ability to translate between programming languages, while domain-specific efforts like the Qiskit Code Assistant [20] and Agent-Q [21] have begun applying LLMs to quantum code generation. The emergence of multi-agent orchestration frameworks [22], [23] and structured reasoning paradigms such as ReAct [24] and Reflexion [25] further suggest that agentic systems can tackle complex, multi-step code transformation tasks that exceed the capability of single-pass generation.

However, applying LLMs to quantum IR translation presents unique challenges distinct from general-purpose code translation. Quantum IR has strict syntactic requirements (e.g., LLVM type annotations, function declaration ordering), subtle semantic constraints (e.g., qubit pointer indexing conventions, measurement recording protocols), and zero tolerance for hallucinated operations that would alter the circuit’s quantum behavior. In practice, smaller open-weight models (8B–13B parameters) that are practical for local deployment frequently produce structurally incomplete output, omitting scaffolding elements required for valid QIR while preserving the correct

This manuscript has been authored by UT-Battelle, LLC, under Contract No. DEAC05000R22725 with the U.S. Department of Energy. The United States Government retains and the publisher, by accepting the article for publication, acknowledges that the United States Government retains a non-exclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this manuscript, or allow others to do so, for the United States Government purposes. The Department of Energy will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan.

gate sequence.

In this paper, we present an agentic translation system that addresses these challenges through a hybrid architecture combining deterministic parsing with LLM-based translation and iterative repair. Our principal contributions are:

- 1) A **three-path translation strategy** that dynamically selects between deterministic, fully agentic, and hybrid repair modes based on dialect recognition and verification outcomes.
- 2) A **dual-backend verification pipeline** that independently executes the source MLIR and generated QIR on separate quantum simulators (PennyLane Catalyst, CUDA Quantum, or other backends via an extensible registry) and QIR Runner, comparing output probability distributions via total variation distance (TVD).
- 3) A **guardrail-assembler pattern** that recovers valid QIR from structurally incomplete LLM output by extracting gate operations and deterministically reassembling them using validated templates.
- 4) A **context engineering** approach that injects gate mappings and QIR templates directly into agent prompts, eliminating the failure modes associated with retrieval-augmented generation (RAG) on smaller models.

The remainder of this paper is organized as follows. Section II reviews related work in quantum IR translation and LLM-based code generation. Section III presents the system architecture and methodology. Section IV describes the experimental setup and evaluation metrics. Section V reports experimental results. Section VI discusses findings and limitations, and Section VII concludes with directions for future work.

II. RELATED WORK

We organize the related work into four areas: quantum IRs, MLIR-based quantum compilation, LLM-driven code generation and translation, and verification of quantum circuit transformations.

A. Quantum Intermediate Representations

The proliferation of quantum hardware platforms has motivated the development of multiple intermediate representations that serve as abstraction layers between high-level quantum programs and low-level control pulses. OpenQASM 3 [26], developed by IBM, extends the original OpenQASM language with classical control flow, subroutines, and timing primitives, establishing itself as a widely adopted circuit-level interchange format. The QIR specification [13], proposed by the QIR Alliance, takes a complementary approach by encoding quantum operations as intrinsic function calls within LLVM IR [14], thereby inheriting LLVM’s mature optimization infrastructure and enabling interoperability with classical compilation pipelines.

Cardama et al. [8] provide a comprehensive survey of quantum IRs, comparing representations including QIR, QSSA, and various MLIR-based dialects along dimensions of expressiveness, optimization support, and tool ecosystem maturity. Ittah et al. [27] proposed QIRO, an SSA-based quantum program

representation built on MLIR that supports dataflow-driven optimizations analogous to those in classical compilers. Luo and Zhao [28] contributed a formalization of quantum intermediate representations, analyzing type-safety properties and demonstrating that well-typed quantum IR programs preserve qubit linearity and measurement consistency. These efforts collectively establish the theoretical foundations upon which our translation system operates, but none address the problem of automated cross-dialect translation.

B. MLIR for Quantum Compilation

The adoption of MLIR [9] for quantum computing was pioneered by McCaskey and Nguyen [11], who demonstrated that MLIR’s progressive lowering mechanism and extensible dialect system are well-suited for expressing quantum operations at multiple levels of abstraction. This foundational insight has been realized in two major quantum compilation frameworks.

PennyLane Catalyst [10] introduces a quantum dialect within MLIR that enables just-in-time compilation of hybrid quantum-classical programs. Catalyst’s dialect defines operations such as `quantum.alloc`, `quantum.custom`, and `quantum.measure` that map directly to quantum gate semantics, and the compiler lowers these progressively through MLIR’s pass infrastructure to executable code. CUDA Quantum [7] defines the Quake dialect, which follows a similar multi-level approach but targets NVIDIA’s GPU-accelerated quantum simulation backends and can lower to QIR as a compilation target.

Recent work by Hopf et al. [12] has explored integrating PennyLane Catalyst with the Munich Quantum Toolkit (MQT) via MLIR, demonstrating that the MLIR ecosystem can serve as a bridge between independently developed quantum tools. While these efforts advance intra-MLIR interoperability, they rely on manually authored compiler passes for each dialect pair, the very scalability limitation our agentic approach is designed to overcome.

C. LLM-Driven Code Generation and Translation

Large language models have achieved strong performance on code generation benchmarks [15], [16], [29], with models such as GPT-4 [19] and Code Llama [18] demonstrating the ability to produce functionally correct programs from natural language specifications. Szafraniec et al. [17] showed that incorporating compiler intermediate representations (specifically LLVM IR) as a pivot improves neural code translation between languages, a finding directly relevant to our MLIR-to-QIR translation task.

In the quantum domain, Dupuis et al. [20] developed the Qiskit Code Assistant by fine-tuning Granite LLMs on quantum computing code, achieving improved performance on Qiskit-specific generation tasks. Jern et al. [21] proposed Agent-Q, which applies fine-tuned LLMs to quantum circuit generation and optimization in OpenQASM 3 format. However, both of these efforts target high-level quantum SDKs

(Qiskit Python or OpenQASM) rather than low-level LLVM-based representations like QIR, which imposes substantially stricter syntactic and structural requirements.

A complementary line of work has explored machine learning for classical compiler optimization. Cummins et al. [30] introduced ProGraML, a graph-based deep learning framework for program optimization using LLVM IR as the representation, demonstrating that neural models can reason about compiler IRs. Our work differs in that we use LLMs for cross-IR *translation* rather than optimization within a single IR, and we operate on quantum-extended LLVM IR where correctness is defined by quantum mechanical equivalence of output distributions.

D. Agentic AI for Software Engineering

The concept of LLM-based agents that combine reasoning with tool use has been formalized by the ReAct paradigm [24], which interleaves chain-of-thought reasoning [31] with actions such as code execution and web search. Shinn et al. [25] extended this approach with Reflexion, which enables agents to improve through verbal self-reflection on execution feedback, a pattern closely related to our iterative verification-feedback loop.

Multi-agent frameworks such as AutoGen [23] and CrewAI [22] provide infrastructure for orchestrating multiple specialized agents on complex tasks. Wang et al. [32] survey the application of LLM-based agents to software engineering, identifying code generation, testing, and debugging as areas of demonstrated success. Our system extends these patterns to quantum compiler engineering, using a translation agent that generates QIR and a verification pipeline that provides structured feedback for iterative repair. The use of context engineering over retrieval-augmented generation (RAG) [33] is a pragmatic adaptation to the failure modes we observed with smaller models.

E. Quantum Circuit Verification

Verifying the correctness of quantum circuit transformations is a recognized challenge in quantum compilation [34]. Bauer-Marquart et al. [35] proposed symQV, a symbolic verification approach that proves equivalence of quantum programs through automated theorem proving. While formally rigorous, symbolic approaches face scalability limitations for circuits with many qubits or complex classical control flow.

Simulation-based verification offers a complementary approach: by executing both the original and transformed circuits on quantum simulators and comparing output distributions, one can detect semantic discrepancies without requiring formal proofs. Total variation distance (TVD) is a standard metric for comparing probability distributions in quantum information theory [36] and provides a symmetric, bounded measure of distributional agreement. Our dual-backend pipeline implements this simulation-based approach with the additional rigor of executing the source MLIR and target QIR on *independent* backends, thereby avoiding shared-mode failures that could mask translation errors.

Table I positions our work relative to existing quantum translation and code generation approaches.

III. METHODOLOGY

This section presents the architecture and algorithms of the agentic MLIR-to-QIR translation system. The system consists of four principal components: dialect-aware parsing, multi-path translation, dual-backend verification, and an LLM output guardrail with deterministic reassembly. Fig. 1 provides an overview of the system architecture and the interaction between components.

A. Dialect Detection and SSA-Aware Parsing

The translation pipeline begins with automatic dialect detection. The system scans the input MLIR source for dialect-specific markers: the presence of `quantum.custom`, `catalyst.launch_kernel`, or `quantum.alloc` tokens identifies the Catalyst dialect [10], while `quake.alloca`, `!quake.veq`, or `quake.h` tokens identify the Quake dialect [7]. If no known markers are found, the dialect is classified as unseen, triggering the agentic translation path.

For recognized dialects, the system employs an SSA-aware parser that traverses the MLIR program in a single pass while maintaining three dictionaries that track the dataflow through the circuit. The first dictionary maps MLIR variable names to physical qubit indices, resolving `quantum.extract` operations that bind SSA values to specific qubits in the allocated register. The second dictionary records constant values declared by `arith.constant` operations, which supply rotation angles for parameterized gates such as RX, RY, and RZ. The third dictionary traces measurement results through the SSA chain, enabling the parser to correctly associate conditional operations (`scf.if` blocks) with their controlling measurement outcomes.

A key parsing challenge arises from multi-result gates. In the Catalyst dialect, a two-qubit gate such as CNOT produces two output SSA values (one per qubit), expressed as `%out : 2 = quantum.custom "CNOT"() %q0, %q1`. The parser resolves both outputs to their respective physical qubit indices by tracking the operand order. The parser emits an ordered operation list that preserves the original gate sequence, measurement positions, and conditional block boundaries, producing an `MLIRCircuit` data structure consumed by both the deterministic QIR generator and the verification pipeline.

B. Three-Path Translation Strategy

The system implements three translation paths, selected dynamically based on dialect recognition and verification outcomes. Algorithm 1 summarizes the routing logic.

Path 1: Deterministic translation. When the detected dialect is recognized, the parser extracts the circuit structure and the QIR generator emits complete LLVM IR by instantiating validated templates. The generator maps each MLIR gate name to its corresponding QIR intrinsic (e.g., `quantum.custom`

TABLE I: Comparison of approaches to quantum code translation and generation. Our system uniquely combines LLM-based translation with dual-backend simulation verification and support for multiple MLIR dialects.

System	Source	Target	LLM-Based	Multi-Agent	Sim. Verification	Extensible to New Dialects
Catalyst Compiler [10]	MLIR (Catalyst)	Machine code	No	No	No	No
CUDA Quantum [7]	MLIR (Quake)	QIR	No	No	No	No
Qiskit Code Assistant [20]	NL / Code	Qiskit Python	Yes	No	No	N/A
Agent-Q [21]	NL	OpenQASM 3	Yes	No	Yes	N/A
Hopf et al. [12]	MLIR	MLIR	No	No	No	Limited
This work	MLIR (any)	QIR	Hybrid	Yes	Dual-backend	Yes

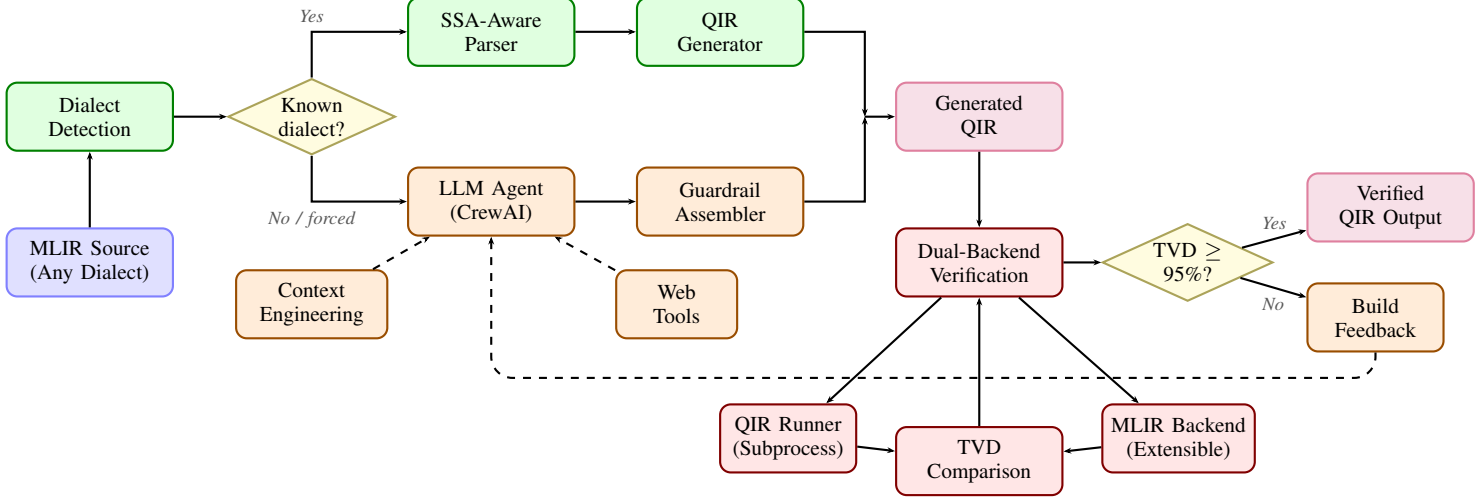


Fig. 1: System architecture of the agentic MLIR-to-QIR translator. Three translation paths are shown: **Path 1** (deterministic, top) for recognized dialects (Catalyst, Quake); **Path 2** (agentic, bottom) for unseen dialects or forced mode; and **Path 3** (repair, dashed) when deterministic output fails verification. The dual-backend verification pipeline executes QIR and MLIR independently on separate simulators (currently PennyLane Catalyst and CUDA Quantum, with an extensible registry for additional backends) and compares distributions via TVD. Dashed arrows indicate feedback and context injection flows.

"Hadamard" to `@__quantum__qis__h__body`), computes qubit pointer expressions using the QIR addressing convention (null for qubit 0, `inttoptr (i64 N to %Qubit*)` for qubit N), and appends measurement operations, output recording calls, function declarations, and QIR metadata. This path requires no LLM invocation and completes in under one second.

Path 2: Fully agentic translation. When the dialect is unseen or the user explicitly forces agentic mode, the system delegates translation to an LLM agent orchestrated through the CrewAI framework [22]. The agent receives the MLIR source along with an inline reference containing gate mapping tables, a complete QIR example, and translation pattern descriptions. This context engineering approach injects all necessary domain knowledge directly into the prompt rather than relying on retrieval-augmented generation (RAG) [33], which we found unreliable with smaller models that frequently misuse retrieval tools. The agent is instructed to produce output in one of two formats: raw LLVM IR (preferred) or a JSON object containing an operations list with qubit indices, which the guardrail can then assemble into valid QIR (Section III-D). When the dialect is unseen, the agent also has access to a

web search tool for fetching documentation about unfamiliar MLIR namespace prefixes.

Path 3: Deterministic with agentic repair. When the deterministic path produces QIR that fails verification (Section III-C), the system transitions to an iterative repair loop. The verification failure is converted into structured feedback that specifies which gates are missing or extraneous, whether the output distribution diverges from the expected reference, and the measured TVD similarity score. This feedback, together with the previous QIR attempt, is provided to the LLM agent, which generates a corrected translation. The loop continues for up to K iterations (configurable, default $K = 5$), tracking the highest-scoring attempt to ensure the system returns its best result even if the maximum iteration count is reached without full convergence. This iterative refinement pattern is inspired by the Reflexion framework [25], adapted from natural language tasks to quantum IR translation.

Table II shows the gate mapping used for both deterministic translation and context engineering in the agentic path.

C. Dual-Backend Verification Pipeline

A central contribution of this work is the dual-backend verification pipeline, which provides an execution-based correct-

Algorithm 1 Translation Path Selection and Verification Loop

Require: MLIR source code M , maximum iterations K , similarity threshold τ

Ensure: QIR code Q , verification result V

```

1:  $d \leftarrow \text{DETECTDIALECT}(M)$ 
2: if  $d \in \{\text{catalyst}, \text{quake}\}$  and not  $\text{force\_agentic}$  then
3:    $Q \leftarrow \text{DETERMINISTICTRANSLATE}(M, d)$  {Path 1}
4:    $V \leftarrow \text{VERIFY}(M, Q)$ 
5:   if  $V.\text{passed}$  then
6:     return  $Q, V$  {Deterministic success}
7:   end if
8:   go to AGENTICLOOP {Path 3: repair with agentic}
9: else
10:  go to AGENTICLOOP {Path 2: agentic}
11: end if
12: AGENTICLOOP:
13:  $Q_{\text{best}} \leftarrow Q$ ,  $s_{\text{best}} \leftarrow V.\text{similarity}$ 
14: for  $i = 1$  to  $K$  do
15:    $F \leftarrow \text{BUILDFEEDBACK}(V)$ 
16:    $Q \leftarrow \text{AGENTTRANSLATE}(M, d, F, Q_{\text{best}}, i)$ 
17:    $Q \leftarrow \text{GUARDRAIL}(Q)$  {Reassemble if needed}
18:    $V \leftarrow \text{VERIFY}(M, Q)$ 
19:   if  $V.\text{gate\_match}$  and  $V.\text{similarity} \geq \tau$  then
20:     return  $Q, V$ 
21:   end if
22:   if  $V.\text{similarity} > s_{\text{best}}$  then
23:      $Q_{\text{best}} \leftarrow Q$ ,  $s_{\text{best}} \leftarrow V.\text{similarity}$ 
24:   end if
25: end for
26: return  $Q_{\text{best}}, V$ 

```

TABLE II: MLIR gate name to QIR intrinsic mapping. Single-qubit gates map to unary QIR functions; two-qubit gates map to binary functions with control and target qubit arguments.

MLIR Gate	QIR Intrinsic Suffix	Params
Hadamard	@__quantum__qis__h__body	—
PauliX	@__quantum__qis__x__body	—
PauliY	@__quantum__qis__y__body	—
PauliZ	@__quantum__qis__z__body	—
S	@__quantum__qis__s__body	—
T	@__quantum__qis__t__body	—
RX	@__quantum__qis__rx__body	θ
RY	@__quantum__qis__ry__body	θ
RZ	@__quantum__qis__rz__body	θ
CNOT	@__quantum__qis__cnot__body	—
CZ	@__quantum__qis__cz__body	—
SWAP	@__quantum__qis__swap__body	—

ness check that goes beyond syntactic or structural validation. The pipeline operates in four stages, each progressively more computationally expensive. Fig. 2 illustrates the verification flow.

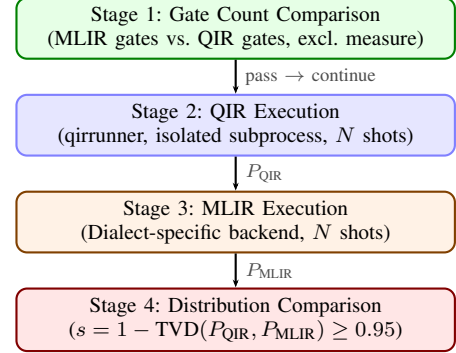


Fig. 2: Four-stage dual-backend verification pipeline. Each stage provides progressively deeper correctness assurance, from structural gate comparison to distributional equivalence via TVD.

Stage 1: Gate count comparison. The system independently counts gates in the original MLIR and the generated QIR. For MLIR, gate names are extracted from `quantum.custom` (Catalyst) or dialect-specific operations (Quake) and normalized to lowercase canonical names. For QIR, gate names are extracted from `call void @__quantum__qis__<gate>__body` patterns. Measurement operations are excluded from both counts because they are infrastructure operations injected at verification time rather than unitary gates specified by the circuit designer. A gate count mismatch provides an immediate signal that the translation introduced or dropped operations.

Stage 2: QIR execution. The generated QIR is executed using the `qirrunner` package (version 0.9.1), which provides a Python interface to a QIR execution engine. To avoid conflicts between the LLVM libraries linked by `qirrunner` and the JAX/XLA runtime used by PennyLane Catalyst, execution is performed in an isolated subprocess. Before execution, a preprocessing step ensures that measurement operations cover all qubits in the circuit and that output recording calls are present in the correct order. The executor runs the circuit for N shots (default $N = 1024$) and parses the output into a bitstring frequency distribution P_{QIR} .

Stage 3: MLIR execution. The original MLIR circuit is independently executed on a dialect-specific backend selected through an extensible simulator registry. For the Catalyst dialect, the system reconstructs a PennyLane quantum node (`QNode`) from the parsed circuit, applies the `@catalyst.qjit` just-in-time compiler with the `lightning.qubit` simulator [6], and collects measurement samples. For circuits containing mid-circuit measurements (conditional operations), the system uses a snapshot-based construction approach that replays gate sequences inside the JIT-compiled function using Catalyst’s `@catalyst.cond`

decorator for branching. For the Quake dialect, the system uses CUDA Quantum [7] as the execution backend, constructing a kernel via `cudaq.make_kernel()` and sampling with `cudaq.sample()`. Both backends execute in isolated subprocesses to avoid LLVM library conflicts. The registry pattern allows additional backends to be registered for new dialects; for unseen dialects where no backend is yet available, the pipeline falls back to QIR-only verification (Stages 1, 2, and 4 with the QIR-side distribution only). The execution produces a second bitstring frequency distribution P_{MLIR} .

Stage 4: Distribution comparison. The two output distributions are compared using the total variation distance (TVD), defined as:

$$\text{TVD}(P, Q) = \frac{1}{2} \sum_{x \in \mathcal{X}} |P(x) - Q(x)| \quad (1)$$

where $\mathcal{X}$ is the set of all observed bitstring outcomes across both distributions, and $P(x)$ and $Q(x)$ are the empirical probabilities of outcome x under each distribution. The TVD ranges from 0 (identical distributions) to 1 (completely disjoint support). We convert TVD to a similarity score $s = 1 - \text{TVD}$ and apply a pass threshold of $\tau = 0.95$, meaning the translation is accepted when the output distributions agree to within 5% total variation. This threshold accounts for finite sampling noise inherent in shot-based simulation while remaining sensitive to gate errors that would produce qualitatively different output states.

We choose TVD over Kullback-Leibler (KL) divergence for two reasons. First, TVD is symmetric, which avoids the arbitrary choice of which distribution serves as the reference. Second, TVD is well-defined even when one distribution assigns zero probability to an outcome observed in the other, whereas KL divergence diverges in this case unless smoothing is applied.

D. Guardrail-Assembler Pattern for LLM Output Recovery

Smaller LLMs (8B to 13B parameters) frequently produce output that contains the correct quantum operations but lacks the structural scaffolding required for valid QIR. Common failure modes include omitting type definitions (`%Qubit = type opaque`), function declarations, attribute sections, and QIR metadata. In some cases, models emit a JSON object with an operations list rather than raw LLVM IR, or they produce tool-call JSON when confused by the agent framework’s tool-use protocol.

To recover valid QIR from these partial outputs, we introduce a guardrail-assembler pattern that operates as a post-processing stage after each LLM invocation. The guardrail applies a cascade of five checks in order of priority, as shown in Fig. 3.

The deterministic QIR assembler accepts a list of parsed gate operations and produces syntactically complete LLVM IR by instantiating the same templates used by the deterministic translation path. It normalizes gate names through an alias table that maps common LLM variants (e.g., “hadamard”, “cx”, “pauli_x”) to canonical QIR function suffixes, infers

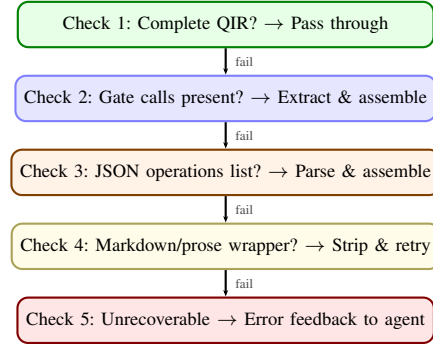


Fig. 3: Guardrail-assembler cascade. Each check attempts to extract valid gate operations from the LLM output using progressively more aggressive parsing strategies.

the qubit count from the maximum qubit index referenced in the operations, and automatically appends measurement operations for all qubits, output recording calls, function declarations, attributes, and QIR version metadata. This design ensures that any LLM output containing correct gate operations, regardless of formatting deficiencies, can be recovered into executable QIR.

E. Context Engineering for LLM Translation

Traditional approaches to providing domain knowledge to LLM agents rely on retrieval-augmented generation (RAG) [33], in which relevant documents are retrieved from a vector database and injected into the prompt at query time. In preliminary experiments, we found that smaller models (8B parameters) frequently failed to use the retrieval tool correctly, issuing repeated identical queries or misinterpreting retrieved content. We therefore adopted a context engineering strategy in which all necessary reference material is injected directly into the task prompt.

The inline reference consists of three components: (1) a gate mapping table that associates each MLIR gate name with its QIR intrinsic function signature (Table II), covering single-qubit gates, parameterized rotation gates, and two-qubit gates; (2) a complete QIR example showing a two-qubit Bell state circuit with all required sections (header, entry function, declarations, attributes, and metadata); and (3) a set of translation pattern descriptions covering qubit allocation, gate sequencing, measurement recording, and conditional operations.

This approach increases prompt length but eliminates the retrieval failure mode entirely. Since the gate mapping table and QIR template are compact (approximately 2,000 tokens), the overhead is acceptable even for models with 4,096-token context windows. For unseen dialects where the inline reference may be insufficient, the agent retains access to a web search tool that can fetch external documentation.

F. Multi-Model Support

The system supports multiple LLM backends through a unified configuration layer. Local models are served through Ollama, supporting Llama 3.1 [37] (8B and 70B variants)

and Code Llama [18] (13B and 34B variants). Cloud-hosted models are accessed through the HuggingFace Inference API. The CrewAI agent framework [22] wraps the selected model in a unified interface for prompt formatting and ReAct [24] reasoning.

IV. EXPERIMENTAL SETUP

A. Benchmark Circuits

We assembled a benchmark suite of 39 unique quantum circuits spanning nine categories, with each circuit available in both Catalyst and Quake MLIR dialects where applicable. Table III summarizes the benchmark. GHZ circuits provide a controlled scaling ladder from 4 to 100 qubits across 20 sizes (every 5 qubits from 5 to 100, plus 4) with identical gate structure (one Hadamard followed by $N-1$ CNOT gates). Measurement-based quantum computing (MBQC) circuits test conditional handling with mid-circuit measurements and classical feed-forward corrections. Random circuits stress-test gate diversity with mixed gate sets including rotations (RX, RY, RZ), phase gates (S, T), and two-qubit entangling operations (CZ, SWAP). The Steane code circuit is an error-correcting code pattern with 7 qubits and 10 CNOT gates. Four FTQC circuits written in an unseen dialect exercise the agentic translation path on Steane-encoded logical operations.

For GHZ circuits at sizes 30 and 100, we generated independent QIR ground-truth files via the `qiskit-qir` package from equivalent Qiskit `QuantumCircuit` objects, providing a third-party reference for structural comparison. The FTQC circuits use a custom `ftqc` dialect with Steane-encoded logical qubit operations (`ftqc.init_zero`, `ftqc.logical_h`, `ftqc.logical_cnot`), which is not registered in the deterministic parser. These circuits exercise the agentic translation path on a genuinely unseen dialect.

B. Models Under Test

We evaluated five LLM models spanning three size classes and two architectures (Table IV). All local models were served through Ollama; the cloud model accessed the HuggingFace Inference API. The deterministic parser (no LLM) served as the baseline for all comparisons.

C. Evaluation Metrics

We report four primary metrics across experiments:

- **Gate match rate:** Binary correctness of gate-level translation. The MLIR and QIR gate counts (excluding measurement operations) must be identical in type and quantity.
- **TVD similarity:** $s = 1 - \text{TVD}(P_{\text{QIR}}, P_{\text{MLIR}})$, measured in two modes: *shots mode* (1,024 samples per backend, both sides subject to sampling noise) and *probs mode* (exact probabilities from MLIR-side state vector via `qml.probs()`, 100K shots on QIR side). The probs mode isolates translation errors from sampling noise.
- **Translation time:** Wall-clock time from MLIR input to QIR output, excluding verification.
- **Iteration count:** Number of agent refinement iterations before convergence (1 for deterministic path).

D. Hardware and Software Configuration

All experiments were conducted on a workstation equipped with an NVIDIA RTX 6000 Ada GPU (48 GB VRAM), Intel Xeon w5-2465X processor, and 256 GB RAM, running Ubuntu 22.04. Software versions: Python 3.12, PennyLane 0.44.0, PennyLane-Catalyst 0.14.0, CUDA Quantum 0.13, qirrunner 0.9.1, CrewAI 1.10.0, Ollama 0.6. We note that qirrunner and CrewAI are under active development; API changes in future releases may require minor adaptation. Each agentic experiment was repeated 3 times to capture LLM output variance.

V. RESULTS

A. Translation Correctness (E1)

Table V reports the deterministic translation results across both dialects and verification modes. The benchmark includes 37 Catalyst circuits and 35 Quake circuits, for a total of 72 dialect-circuit pairs. Of these, 71 achieve correct gate-level translation; the single failure is a Quake-dialect Bell state file misrouted through the Catalyst parser due to its placement in a generic examples directory rather than the dialect-specific directory. Circuits with 35 or more qubits are translated and gate-checked but excluded from TVD statistics because the 2^N -dimensional state vector exceeds available memory for simulation. Among the 38 circuit pairs where both backends could be executed, TVD similarity averages 97.9% in shots mode and 98.8% in probs mode. The gap between modes confirms that the residual TVD shortfall under shots mode is attributable to finite sampling noise rather than translation error. Three circuits (two random circuits and one conditional circuit) exhibit gate-level matches but produce divergent simulation distributions (TVD near 0%), which we attribute to mid-circuit measurement semantics that the MLIR-side executor does not fully reconstruct; these are discussed in Section VI-D.

B. Scalability (E2)

To assess how translation quality scales with circuit size, we evaluated GHZ circuits across seven sizes (5, 10, 15, 20, 25, and 30 qubits with full TVD verification, plus 100 qubits with gate checking only) in both Catalyst and Quake dialects. All 16 circuit-dialect pairs achieve 100% gate match. Deterministic translation time remains below 25 ms across all sizes, confirming $O(n)$ scaling with respect to gate count. Among the 14 pairs where TVD could be computed (GHZ-5 through GHZ-30, both dialects), shots-mode TVD averages 98.5% with a minimum of 96.2%, and probs-mode TVD averages 99.4% with a minimum of 97.3%. GHZ-100 is correctly translated (100 gates, verified by structural comparison against an independent Qiskit-generated QIR reference) but cannot be simulated on 256 GB RAM.

C. Cross-Dialect Portability (E6)

To test whether the system produces semantically equivalent QIR regardless of input dialect, we translated 14 matched Catalyst–Quake circuit pairs through the deterministic path

TABLE III: Benchmark circuit suite (39 unique circuits). Circuits are organized by category with qubit counts, gate types, dialect availability, and count of distinct circuits per row.

Category	Circuit	Qubits	Key Gates	Dialects	Count
Standard	Bell state	2	H, CNOT	Both (Catalyst & Quake)	1
	GHZ-3	3	H, CNOT $\times 2$	Both	1
GHZ scaling	GHZ- $\{5, 10, 15, \dots, 100\}$	5–100	H, CNOT $\times (N-1)$	Both	20
MBQC	Teleport, RZ rotation	2	H, CZ, X, conditional	Both	2
	RX rotation, CNOT	3–4	H, CZ, X, Z, conditional	Both	2
Conditional	Classic teleportation	3	H, CNOT, X, Z	Both	1
Random	5 mixed-gate circuits	2–7	T, S, RX, RY, RZ, CZ, SWAP	Both	5
QEC	Steane code	7	H $\times 10$, CNOT $\times 9$	Catalyst only	1
Parametric	RX/RY/RZ rotations	1	RX, RY, RZ	Catalyst only	1
Variational	Autodiff gradient	1	RX, ExpVal(PauliZ)	Catalyst only	1
FTQC (unseen)	Steane 1Q H, 2Q Bell	1–2 [†]	Logical H, CNOT	FTQC only	2
	Steane 3Q Grover, 3Q QFT	3 [†]	Logical H, CZ, Z	FTQC only	2
Total unique circuits					39

[†]Logical qubits; physical count is $7\times$ under Steane $[[7,1,3]]$ encoding.

TABLE IV: LLM models evaluated. VRAM listed for local Ollama deployment.

Model	Provider	Params	VRAM
Llama 3.1	Ollama (local)	8B	~6 GB
Llama 3.1	Ollama (local)	70B	~40 GB
Code Llama	Ollama (local)	13B	~8 GB
Code Llama	Ollama (local)	34B	~20 GB
GPT-OSS-20B	HuggingFace (API)	20B	Cloud

TABLE V: E1: Deterministic translation correctness. Gate match rate and TVD similarity across dialects and verification modes. Circuits ≥ 35 qubits are gate-checked only (TVD requires state-vector simulation). Three circuits with known simulation divergence and one misrouted dialect file are excluded from TVD averages.

Dialect	Mode	Total	Gate OK	TVD n	Avg	Min
Catalyst	Shots	37	36	20	97.7%	91.3%
Catalyst	Probs	37	36	20	99.3%	96.3%
Quake	Shots	35	35	18	98.2%	93.4%
Quake	Probs	35	35	18	98.4%	93.8%

and compared the outputs. All 14 pairs achieve gate-level match in both dialects. For the 11 pairs where TVD could be computed on both sides, Catalyst-side TVD averages 99.4% and Quake-side TVD averages 98.5%, both in probs mode. Two random circuits (`random_1847` and `random_2449`) produce TVD near 0% despite correct gate counts, the same simulation-divergence issue observed in E1. GHZ-100 is gate-matched on both sides but excluded from TVD comparison.

D. Unseen Dialect Translation (E7)

To evaluate extensibility beyond the two registered dialects, we tested the agentic path on four circuits written in a custom

FTQC dialect that encodes logical qubits using the Steane $[[7,1,3]]$ error-correcting code. The FTQC dialect uses operations such as `ftqc.init_zero` and `ftqc.logical_h` that have no entry in the deterministic parser’s gate table. When the system encounters this dialect, it identifies the `ftqc` namespace as unknown and routes the circuit to the agentic translation path, equipping the LLM agent with web search and simulator discovery tools.

Table VI reports the results. The deterministic path rejects all four circuits with exit code 2 (unsupported dialect), confirming correct routing. Llama 3.1 (8B) produces valid QIR for all four circuits on the first iteration, mapping each logical FTQC operation to its QIR counterpart (e.g., `ftqc.logical_h` $\rightarrow$ `@__quantum__qis__h__body`). Code Llama (13B) succeeds on the two simpler circuits but fails on the three-qubit Grover and QFT circuits, emitting tool-call JSON instead of LLVM IR. This failure mode is specific to code-specialized models whose tool-use training interferes with direct output generation in the CrewAI agent framework.

The generated QIR represents a logical-level translation: the LLM correctly identifies the gate semantics of the FTQC operations but does not expand them into the physical Steane encoding, which would require 7 transversal gates per logical operation and a 12-gate encoding circuit per logical qubit. This distinction between logical and physical QIR is inherent to fault-tolerant quantum computing. In current FTQC compilation stacks, the error-correction encoding is typically the responsibility of the backend runtime rather than the IR, making the logical-level output a reasonable translation target.

VI. DISCUSSION

A. Deterministic vs. Agentic Trade-offs

The deterministic path translates recognized dialects in under 25 ms with near-perfect gate-level correctness (71/72

TABLE VI: E7: Unseen dialect (FTQC) translation. Success indicates valid QIR produced; logical gates lists the gate calls in the generated QIR. No MLIR backend exists for FTQC, so TVD verification is not applicable. Each circuit was run 3 times per model.

Circuit	Model	Success	Iter.	Logical Gates
1Q Hadamard	Llama 3.1 (8B)	3/3	1	H, mz
	Code Llama (13B)	3/3	1	H, mz
2Q Bell	Llama 3.1 (8B)	3/3	1	H, CNOT, 2×mz
	Code Llama (13B)	3/3	1	H, CNOT, 2×mz
3Q Grover	Llama 3.1 (8B)	3/3	1	2H, 2CNOT, 6×mz
	Code Llama (13B)	0/3	–	(tool-call JSON)
3Q QFT	Llama 3.1 (8B)	3/3	1	3H, 2CNOT, CZ
	Code Llama (13B)	0/3	–	(tool-call JSON)

circuit pairs). The agentic path trades speed for generality: it handles unseen dialects at the cost of 5–15 seconds per translation and model-dependent output quality. The repair path (Path 3) combines both approaches, using the deterministic parser as a first attempt and falling back to LLM repair only when verification fails. In practice, the repair path is triggered by circuits whose conditional logic the deterministic parser handles structurally but that produce unexpected simulation behavior due to mid-circuit measurement semantics.

B. Context Engineering vs. RAG

Replacing RAG with inline context injection was driven by failure modes observed with 8B-parameter models. These models issued repeated identical retrieval queries, misinterpreted retrieved passages, or produced tool-call JSON instead of translation output. Injecting the gate mapping table and QIR template directly into the prompt (~2,000 tokens) eliminated retrieval failures while keeping the overhead within the context window of even small models. The agent retains access to web search tools for unseen dialects, though our experiments indicate that current models rarely use these tools without explicit instruction.

C. Unseen Dialect Generalization

The FTQC experiment (E7) shows that the system translates circuits from entirely new dialects without code changes. The LLM agent infers logical gate semantics from operation names alone (`ftqc.logical_h` → Hadamard), producing valid QIR that captures the circuit’s logical structure. The gap between logical-level and physical-level translation (Steane encoding expansion) reflects domain knowledge absent from the LLM’s training data. An unexpected finding is that the general-purpose Llama 3.1 (8B) outperforms the code-specialized Code Llama (13B) on this task: Code Llama’s tool-use training causes it to emit tool-call JSON rather than LLVM IR when processing unfamiliar dialects. Bridging the logical-to-physical gap could involve few-shot examples of error-correction encodings, domain-specific retrieval from QEC documentation, or models fine-tuned on fault-tolerant compilation patterns.

D. Limitations

Several limitations constrain the current evaluation. First, simulation-based verification requires 2^N memory for an N -qubit circuit; circuits with 35 or more qubits can only be gate-checked, not distribution-compared. Second, three circuits (`random_1847`, `random_2449`, and one conditional circuit) produce correct gate counts but divergent output distributions (TVD near 0%). In each case, the MLIR-side executor does not fully reconstruct the mid-circuit measurement and classical feed-forward logic, leading to a mock or degenerate distribution. The QIR-side execution produces the expected output, indicating a limitation of the MLIR backend reconstruction rather than a translation error. Third, the autodiff gradient circuit is not supported in the verification pipeline because the MLIR-side executor does not reconstruct gradient computation graphs. Fourth, one Quake-dialect Bell state file placed in a generic directory was misrouted through the Catalyst parser, producing an empty QIR output; this is a file-organization issue rather than a parser defect. Fifth, for the unseen FTQC dialect, no MLIR backend is available, so verification is limited to structural inspection of the generated QIR. Finally, our benchmark covers two registered dialects and one unseen dialect; evaluation across additional MLIR dialects (OpenPulse, Quil, Stim) would test generalizability more broadly.

VII. CONCLUSION

We presented a hybrid system for MLIR-to-QIR translation that integrates deterministic compiler techniques with LLM-based agentic translation and iterative repair. The three-path architecture routes circuits through the fastest reliable path: deterministic parsing for known dialects, agentic translation for unseen dialects, and LLM-guided repair when verification fails. A dual-backend verification pipeline provides execution-based correctness checking by running source MLIR and target QIR on independent simulators and comparing output distributions via TVD.

On a 39-circuit benchmark spanning two registered dialects (Catalyst and Quake) and one unseen dialect (FTQC), with GHZ circuits scaling from 5 to 100 qubits across 20 sizes, the deterministic path achieved 98.6% gate-level correctness (71/72 pairs) with TVD similarity averaging 97.9–99.3% across 38 simulable circuit pairs. Cross-dialect evaluation on 14 matched Catalyst–Quake pairs confirmed gate-level equivalence in all cases. For the unseen FTQC dialect, the agentic path produced valid logical-level QIR for all four test circuits using Llama 3.1 (8B), demonstrating that the system generalizes to dialects absent from its deterministic parser.

Future work will extend the system along four axes: supporting additional MLIR dialects as they emerge in the quantum ecosystem, fine-tuning domain-specific LLMs on a corpus of quantum IR translations, integrating formal verification methods alongside simulation-based TVD checking, and extending the verification pipeline to support fault-tolerant encoding expansions and parameterized variational circuits.

ACKNOWLEDGMENT

This work was supported by the U.S. Department of Energy, Office of Science under Contract No. DE-AC05-00OR22725, with funding from the Office of Advanced Scientific Computing Research’s Accelerated Research in Quantum Computing Program’s Modular and Error-Aware Software Stack for Heterogeneous Quantum Computing Ecosystems (MACH-Q) project.

REFERENCES

- [1] J. Preskill, “Quantum computing in the NISQ era and beyond,” *Quantum*, vol. 2, p. 79, 2018.
- [2] B. Heim, M. Soeken, S. Marshall, C. Granade, M. Roetteler, A. Geller, M. Troyer, and K. Svore, “Quantum programming languages,” *Nature Reviews Physics*, vol. 2, no. 12, pp. 709–722, 2020.
- [3] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, “Quantum computing with Qiskit,” *arXiv preprint arXiv:2405.08810*, 2024.
- [4] Cirq Developers, “Cirq,” 2024.
- [5] K. Svore, A. Geller, M. Troyer, J. Azariah, C. Granade, B. Heim, V. Kliuchnikov, M. Mykhailova, A. Paz, and M. Roetteler, “Q#: Enabling scalable quantum computing and development with a high-level DSL,” *Proceedings of the Real World Domain Specific Languages Workshop*, 2018.
- [6] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, M. S. Alam, S. Ahmed, J. M. Arrazola, C. Blank, A. Delgado, S. Jahangiri *et al.*, “PennyLane: Automatic differentiation of hybrid quantum-classical computations,” *arXiv preprint arXiv:1811.04968*, 2022.
- [7] J.-S. Kim, A. McCaskey, B. Heim, M. Modani, S. Stanwyck, and T. Costa, “CUDA Quantum: The platform for integrated quantum-classical computing,” in *Proceedings of the 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–4.
- [8] F. J. Cardama, J. Vázquez-Pérez, C. Piñeiro, J. C. Pichel, T. F. Pena, and A. Gómez, “Review of intermediate representations for quantum computing,” *The Journal of Supercomputing*, vol. 81, no. 418, 2025.
- [9] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shepman, N. Vasilache, and O. Zinenko, “MLIR: Scaling compiler infrastructure for domain specific computation,” in *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021, pp. 2–14.
- [10] D. Ittah, A. Asadi, E. O. Lopez, S. Mironov, S. Banning, R. Moyard, M. J. Peng, and J. Izaac, “Catalyst: a Python JIT compiler for auto-differentiable hybrid quantum programs,” vol. 9, no. 99, 2024, p. 6720.
- [11] A. McCaskey and T. Nguyen, “A MLIR dialect for quantum assembly languages,” in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2021, pp. 255–264.
- [12] P. Hopf, E. O. Lopez, Y. Stade, D. Rovara, N. Quetschlich, I. A. Florea, J. Izaac, R. Wille, and L. Burgholzer, “Integrating quantum software tools with(in) MLIR,” in *Proceedings of Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region*, 2026, arXiv:2601.02062.
- [13] QIR Alliance, “QIR specification: Quantum intermediate representation,” <https://github.com/qir-alliance/qir-spec>, 2021, accessed: 2026-03-01.
- [14] C. Lattner and V. Adve, “LLVM: A compilation framework for lifelong program analysis & transformation,” in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2004, pp. 75–86.
- [15] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [16] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. D. Lago *et al.*, “Competition-level code generation with AlphaCode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [17] M. Szafraniec, B. Rozière, H. Leather, F. Charton, P. Labatut, and G. Synnaeve, “Code translation with compiler representations,” in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2207.03578.
- [18] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez *et al.*, “Code Llama: Open foundation models for code,” *arXiv preprint arXiv:2308.12950*, 2023.
- [19] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, “GPT-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [20] N. Dupuis, L. Buratti, S. Vishwakarma, A. V. Forrat, D. Kremer, I. Faro, R. Puri, and J. Cruz-Benito, “Qiskit code assistant: Training LLMs for generating quantum computing code,” in *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 2024, pp. 1–4.
- [21] L. Jern, V. Uotila, C. Yu, and B. Zhao, “Agent-Q: Fine-tuning large language models for quantum circuit generation and optimization,” in *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, 2025.
- [22] J. Moura, “CrewAI: Framework for orchestrating role-playing autonomous AI agents,” <https://github.com/crewAIInc/crewAI>, 2024, version 1.10.0.
- [23] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [24] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [25] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 8634–8652, 2023.
- [26] A. W. Cross, A. Javadi-Abhari, T. Alexander, N. de Beaudrap, L. S. Bishop, S. Heide, C. A. Ryan, P. Sivarajah, J. Smolin, J. M. Gambetta, and B. R. Johnson, “OpenQASM 3: A broader and deeper quantum assembly language,” *ACM Transactions on Quantum Computing*, vol. 3, no. 3, pp. 1–50, 2022.
- [27] D. Ittah, T. Häner, V. Kliuchnikov, and T. Hoefler, “QIRO: A static single assignment-based quantum program representation for optimization,” *ACM Transactions on Quantum Computing*, vol. 3, no. 3, pp. 1–32, 2022.
- [28] J. Luo and J. Zhao, “Formalization of quantum intermediate representations,” 2023, arXiv:2303.14500.
- [29] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, “Program synthesis with large language models,” *arXiv preprint arXiv:2108.07732*, 2021.
- [30] C. Cummins, Z. V. Fisches, T. Ben-Nun, T. Hoefler, and H. Leather, “ProGraML: Graph-based deep learning for program optimization and analysis,” *arXiv preprint arXiv:2003.10536*, 2020.
- [31] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [32] J. Wang *et al.*, “Large language model-based agents for software engineering: A survey,” *arXiv preprint arXiv:2409.02977*, 2024.
- [33] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [34] K. Karuppasamy, V. Puram, S. Johnson, and J. P. Thomas, “A comprehensive review of quantum circuit optimization: Current trends and future directions,” *Quantum Reports*, vol. 7, no. 1, p. 2, 2025.
- [35] F. Bauer-Marquart, S. Leue, and C. Schilling, “symQV: Automated symbolic verification of quantum programs,” in *Formal Methods: 25th International Symposium (FM 2023)*, ser. Lecture Notes in Computer Science, vol. 14000. Springer, 2023, pp. 181–198.
- [36] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.
- [37] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman *et al.*, “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.